

Programowanie 2

Obszerna notatka do nauki od podstaw

SQL • pandas • NumPy • scikit-learn • uczenie maszynowe • sieci neuronowe

Materiał opracowany na podstawie repozytorium **KGN_Programowanie2** (laboratoria i wykłady). Notatka prowadzi czytelnika od zupełnych podstaw, tłumacząc każde pojęcie krok po kroku i ilustrując je komentowanymi przykładami kodu.

Spis treści

1. Podstawy języka SQL
2. SQL w praktyce pakietu pandas
3. pandas: Series, DataFrame, loc/iloc, informacje o tabeli
4. NumPy: kształt obiektu ndarray i jego zmiana
5. NumPy: broadcasting, wektoryzacja, wydajność, axis
6. NumPy: referencje (widoki) kontra kopie
7. Czyszczenie i standaryzacja danych
8. Klasyfikacja i regresja w scikit-learn
9. Uczenie z nadzorem i bez nadzoru
10. Miary błędu: MSE, RMSE, MAE, accuracy, FP/FN
11. Przeuczenie (overfitting) i jak mu zapobiegać
12. Podział danych: zbiór treningowy i testowy
13. Perceptron i funkcje aktywacji
14. Wsteczna propagacja błędu (backpropagation)

1. Podstawy języka SQL

SQL (ang. **Structured Query Language**) to język zapytań do relacyjnych baz danych. Baza relacyjna przechowuje dane w **tabelach** — prostokątnych strukturach złożonych z **kolumn** (atrybutów, np. imię, ocena) i **wierszy** (rekordów, czyli pojedynczych krotek danych, np. konkretnego studenta). SQL pozwala te dane pobierać, filtrować, łączyć, grupować i podsumowywać, nie mówiąc komputerowi „jak” to zrobić, a jedynie „co” chcemy uzyskać (jest to język deklaracyjny).

1.1. Budowa (struktura) kwerendy SQL

Kwerenda (zapytanie) to pojedyncze polecenie pobrania danych. Najważniejsze polecenie to **SELECT**. Pełny szkielet zapytania wybierającego dane wygląda następująco:

Ogólny szkielet kwerendy SQL (komentarze po -- nie są wykonywane).

```
SELECT nazwy_i_definicje_kolumn -- CO chcemy zobaczyć (które kolumny)
FROM   nazwy_tabel_i_ich_złączenia -- SKĄD bierzemy dane (z których tabel)
WHERE  warunek_selekcji_wierszy -- KTÓRE wiersze zostawić (filtr)
GROUP BY kolumny_grupujące -- jak POGRUPOWAĆ wiersze w paczki
HAVING warunek_po_grupowaniu -- filtr działający NA GRUPACH
ORDER BY kolumny_sortowania [ASC|DESC]; -- jak POSORTOWAĆ wynik
```

Uwaga: Klauzule WHERE, GROUP BY, HAVING i ORDER BY są opcjonalne. Wymagane jest jedynie SELECT (co wybrać) oraz najczęściej FROM (skąd). Średnik ; kończy zapytanie.

1.2. Słowa kluczowe i ich znaczenie

Słowo kluczowe	Znaczenie / co robi
SELECT	Określa, które kolumny (lub wyrażenia) znajdują się w wyniku. SELECT * oznacza „wszystkie kolumny”.
FROM	Wskazuje tabelę lub tabele będące źródłem danych; tutaj definiujemy też złączenia tabel.
WHERE	Filtruje pojedyncze wiersze — zostają tylko te, dla których warunek logiczny jest prawdziwy.
GROUP BY	Grupuje wiersze o tej samej wartości wskazanej kolumny w jedną „paczke”, na której liczymy funkcje agregujące.
HAVING	Filtruje już utworzone grupy (np. zestaw grupy mające średnią > 3). Działa po GROUP BY.
ORDER BY	Sortuje wynik rosnąco (ASC, domyślnie) lub malejąco (DESC) po wskazanych kolumnach.

Uwaga: W treści zagadnień pojawia się skrót „SORT” — w standardzie SQL klauzula sortująca nosi nazwę ORDER BY i to jej należy używać.

1.3. Kolejność WYKONYWANIA klauzul (logiczna)

Zapytanie zapisujemy zaczynając od SELECT, ale baza danych **wykonuje** je w innej kolejności. Zrozumienie tej kolejności jest kluczowe, bo tłumaczy np. dlaczego w WHERE nie można jeszcze używać aliasów kolumn z SELECT:

- FROM — najpierw tworzone jest jedno wspólne źródło danych (łączenie tabel w jedną).
- WHERE — odrzucane są wiersze niespełniające warunku (zanim cokolwiek pogrupujemy).
- GROUP BY — pozostałe wiersze są grupowane po wskazanych atrybutach.
- HAVING — odrzucane są całe grupy na podstawie wartości funkcji grupujących.
- SELECT — wyliczane są ostateczne kolumny wyniku (i aliasy).
- ORDER BY — na samym końcu wynik jest sortowany.

1.4. Funkcje grupujące (agregujące)

Funkcje agregujące związają wiele wierszy w jedną liczbę. Używamy ich najczęściej razem z GROUP BY:

- count(*) — zlicza WSZYSTKIE wiersze w grupie (także te z wartościami NULL).
- count(atrybut) — zlicza tylko wiersze, w których atrybut jest różny od NULL.
- avg(atrybut) — średnia arytmetyczna wartości w kolumnie.
- max(atrybut) / min(atrybut) — wartość największa / najmniejsza.
- sum(atrybut) — suma wartości w kolumnie.

Uwaga: NULL to brak wartości (nie to samo co 0 ani pusty napis). Większość funkcji agregujących po prostu pomija wartości NULL.

1.5. Złączenia tabel (JOIN)

Dane bywają rozbite na kilka tabel (np. osobno studenci, osobno oceny). Złączenie (JOIN) łączy wiersze różnych tabel w jeden wiersz wyniku na podstawie wspólnej kolumny (klucza). Poniżej najważniejsze typy złączeń:

Typ złączenia	Działanie
NATURAL JOIN	Łączy automatycznie po WSPÓLNYCH kolumnach (o tej samej nazwie). Bierze tylko krotki o zgodnych wartościach w tych kolumnach.
JOIN / INNER JOIN	Łączy po jawnie podanym warunku: T1 JOIN T2 ON T1.A = T2.B. W wyniku zostają tylko pasujące pary wierszy.
LEFT OUTER JOIN	Wszystkie wiersze z LEWEJ tabeli; brakujące dopasowania z prawej uzupełniane są wartościami NULL.
RIGHT OUTER JOIN	Analogicznie — wszystkie wiersze z PRAWEJ tabeli; braki z lewej uzupełniane NULL-ami.
FULL OUTER JOIN	Wszystkie wiersze z OBU tabel; niepasujące dopasowania po obu stronach wypełniane NULL.
CROSS JOIN	Iloczyn kartezjański — każdy wiersz z jednej tabeli łączony z każdym wierszem drugiej (bez warunku).

CROSS JOIN można zapisać też przez przecinek — oba zapisy są równoważne:

Dwa równoważne zapisy iloczynu kartezjańskiego.

```
SELECT * FROM A CROSS JOIN B;    -- iloczyn kartezjański, zapis jawny
SELECT * FROM A, B;              -- dokładnie to samo, zapis skrócony
```

Uwaga: Słowo OUTER jest opcjonalne: LEFT JOIN znaczy to samo co LEFT OUTER JOIN. Jeśli w INNER JOIN zapomnimy warunku ON, baza policzy iloczyn kartezjański — zwykle to błąd.

1.6. Przykład z repozytorium (plik P2Lab02_SQL.ipynb)

W laboratorium dane są tworzone w pandas i zapisywane do bazy SQLite, a następnie odpytywane magią `%%sql`. Najpierw przygotowanie tabel:

Tworzenie tabel w bazie SQLite na podstawie ramek pandas.

```
import pandas as pd
import sqlite3 # wbudowana, plikowa baza danych SQLite

# Tabela studentów: id, imię, kierunek
df_studenci = pd.DataFrame({
    'id_stud': [101, 102, 103, 104],
    'imie': ['Adam', 'Beata', 'Cezary', 'Daria'],
    'kierunek': ['Filozofia', 'Kognitywistyka', 'Filozofia', 'Informatyka']
})

# Tabela ocen: 105 to ocena studenta spoza listy studentów (celowo!)
df_oceny = pd.DataFrame({
    'id_stud': [101, 101, 102, 103, 103, 105],
    'przedmiot': ['Logika I', 'Etyka', 'Logika I', 'Logika I', 'Metafizyka', 'Logika I'],
    'ocena': [5.0, 4.0, 4.5, 3.0, 3.5, 2.0]
})

conn = sqlite3.connect('uczelnia.db') # otwiera/tworzy plik bazy
# zapisuje DataFrame jako tabelę 'studenci'; replace = nadpisz, jeśli istnieje
df_studenci.to_sql('studenci', conn, index=False, if_exists='replace')
df_oceny.to_sql('oceny', conn, index=False, if_exists='replace')
conn.close() # zamyka połączenie z bazą
```

Następnie zapytanie łączące studentów z ocenami i liczące statystyki dla każdego studenta. Użyto `LEFT JOIN`, dzięki czemu student bez ocen (Daria, id 104) także pojawi się w wyniku:

Złączenie `LEFT JOIN` + grupowanie + funkcje agregujące.

```
%%sql
SELECT
    s.imie,
    s.kierunek,
    COUNT(o.ocena) AS liczba_egzaminow, -- ile ocen ma student
    ROUND(AVG(o.ocena), 2) AS srednia -- średnia, zaokrąglona do 2 miejsc
FROM studenci s -- s = alias tabeli studenci
LEFT JOIN oceny o ON s.id_stud = o.id_stud -- dołącz oceny po wspólnym id
GROUP BY s.id_stud; -- jeden wiersz na studenta
```

Przykład podzapytania (zapytania zagnieżdżonego) z repozytorium — ile przedmiotów **NIE** może prowadzić dany wykładowca. `NOT IN` sprawdza przynależność do zbioru zwróconego przez wewnętrzne `SELECT`:

Podzapytanie skorelowane z operatorem `NOT IN`.

```
%%sql
SELECT w.nazwisko, COUNT(DISTINCT g.przedmiot) -- DISTINCT = licz różne przedmioty
FROM grafik g JOIN wykładowcy w
WHERE w.id_wykl NOT IN (
    SELECT g2.id_wykl FROM grafik g2 -- wykładowca, którego NIE ma
    WHERE g2.przedmiot = g.przedmiot -- wśród prowadzących dany przedmiot
)
GROUP BY w.nazwisko
ORDER BY w.nazwisko;
```

2. SQL w praktyce pakietu pandas

pandas to biblioteka Pythona do pracy z danymi tabelarycznymi. Każda operacja SQL ma swój odpowiednik w pandas. Poniżej omawiamy je po kolei, korzystając z tych samych tabel (`df_studenci`, `df_oceny`) co w sekcji 1.

2.1. Jak wykonać kwerendę SQL w pandas?

Są trzy podejścia. (a) Napisać prawdziwy SQL do bazy i wczytać wynik funkcją `pd.read_sql`; (b) użyć metody `.query()` na ramce; (c) odtworzyć logikę zapytania metodami pandas. Najpierw wariant z prawdziwym SQL:

Wariant (a): pełny SQL wykonany w bazie, wynik jako DataFrame.

```
import pandas as pd, sqlite3
conn = sqlite3.connect('uczelnia.db')
# pd.read_sql wykonuje zapytanie w bazie i zwraca wynik od razu jako DataFrame
wynik = pd.read_sql('SELECT * FROM studenci WHERE kierunek = "Filozofia"', conn)
conn.close()
print(wynik)
```

Wariant (b): `.query()` — odpowiednik klauzuli WHERE.

```
# Wariant (b): metoda .query() przyjmuje warunek w stylu SQL/WHERE jako napis
df_studenci.query('kierunek == "Filozofia"')
```

2.2. Selekcja wierszy warunkiem logicznym (WHERE)

To tzw. **indeksowanie logiczne (boolean masking)**. W nawiasie kwadratowym podajemy warunek, który dla każdego wiersza daje True/False; zostają tylko wiersze z True:

Filtrowanie wierszy maską logiczną — odpowiednik WHERE.

```
# Zostaw studentów z kierunku Filozofia (odpowiednik: WHERE kierunek = 'Filozofia')
df_studenci[df_studenci['kierunek'] == 'Filozofia']

# Warunek złożony: oceny >= 4 ORAZ przedmiot Logika I.
# & to logiczne I, | to logiczne LUB. KAŻDY warunek MUSI być w nawiasach!
df_oceny[(df_oceny['ocena'] >= 4) & (df_oceny['przedmiot'] == 'Logika I')]
```

Uwaga: W pandas używamy `&`, `|`, `~` (nie `and/or/not`) i obowiązkowo otaczamy każdy człon nawiasami, bo operatory `&` i `|` mają wyższy priorytet niż porównania.

2.3. Instrukcja grupująca (GROUP BY)

Metoda `.groupby()` dzieli ramkę na grupy, a następnie na każdej grupie liczymy agregat. Przykład wprost z repozytorium — średnia i liczba ocen każdego studenta:

GROUP BY + agregacja w pandas (P2Lab03_pandas.ipynb).

```
# .merge = złączenie (JOIN) studentów z ocenami po kolumnie id_stud, typ left
# .groupby = pogrupuj po imieniu
# .agg = policz na kolumnie 'ocena' dwie funkcje: licznosc i srednia
df1 = (df_studenci
        .merge(df_oceny, on='id_stud', how='left')
        .groupby('imie')['ocena']
        .agg(['count', 'mean']))
print(df1)
```

Agregacja całości oraz grupowanie po przedmiocie.

```
# Średnia WSZYSTKICH ocen (agregacja bez grupowania) — jak SELECT AVG(ocena)
print(f"Średnia ocen to: {df_oceny['ocena'].mean()}")

# Średnia ocen z każdego PRZEDMIOTU osobno (GROUP BY przedmiot)
df_oceny.groupby('przedmiot')['ocena'].mean()
```

2.4. Wybór kolumn (SELECT)

Rzutowanie na wybrane kolumny — odpowiednik `SELECT` imię, kierunek.

```
df_studenci['imię']          # jedna kolumna -> obiekt Series
df_studenci[['imię', 'kierunek']] # lista kolumn -> DataFrame (dwa nawiasy!)
```

Uwaga: Pojedyncze nawiasy zwracają Series (jedną kolumnę), podwójne (lista) zwracają DataFrame.

2.5. Złączenie tabel (JOIN)

W pandas złączeń dokonujemy metodą `.merge()`. Parametr `on` wskazuje wspólną kolumnę, a `how` określa typ złączenia i wprost odpowiada SQL-owym JOIN-om:

pandas (how=...)	Odpowiednik SQL
how='inner'	INNER JOIN (tylko pasujące pary — wartość domyślna)
how='left'	LEFT OUTER JOIN
how='right'	RIGHT OUTER JOIN
how='outer'	FULL OUTER JOIN
how='cross'	CROSS JOIN (iloczyn kartezjański)

Złączenie ramek metodą `.merge()`.

```
# LEFT JOIN: wszyscy studenci + ich oceny (student bez ocen też zostanie, z NaN)
df_studenci.merge(df_oceny, on='id_stud', how='left')
```

2.6. Usuwanie kolumn

Usuwanie kolumn metodą `.drop()` (ostatnia linia z `P2Lab09_housing`).

```
# axis=1 oznacza 'działaj na kolumnach' (axis=0 = wiersze)
df_studenci.drop(columns=['kierunek']) # zwraca NOWĄ ramkę bez kolumny
df_studenci.drop('kierunek', axis=1)   # zapis równoważny

# inplace=True zmienia ramkę „w miejscu” i nic nie zwraca (przykład z repo):
strat_train_set.drop(['income_cat'], axis=1, inplace=True)
```

2.7. Nowa kolumna jako wynik funkcji na kolumnach

Najczęściej tworzymy kolumnę przez wektorowe działanie na istniejących kolumnach albo przez `.apply()` (funkcja stosowana wiersz po wierszu):

Trzy sposoby definiowania nowej kolumny.

```
# (a) Operacja wektorowa – działa od razu na całej kolumnie (szybkie):
df_oceny['ocena_proc'] = df_oceny['ocena'] / 5.0 * 100 # ocena w procentach

# (b) Funkcja własna na każdej wartości przez .apply():
df_oceny['zdany'] = df_oceny['ocena'].apply(lambda o: 'tak' if o >= 3.0 else 'nie')

# (c) Złożony warunek między kolumnami przez np.where:
import numpy as np
df_oceny['status'] = np.where(df_oceny['ocena'] >= 4.5, 'wyróżnienie', 'zwykły')
```

3. pandas: Series, DataFrame, loc/iloc, informacje o tabeli

3.1. Czym jest Series, czym DataFrame i czym się różnią

Series to jednowymiarowa, etykietowana tablica — pojedyncza kolumna danych wraz z indeksem (etykietami wierszy). **DataFrame** to dwuwymiarowa tabela: zbiór kolumn, z których każda jest obiektem Series, połączonych wspólnym indeksem. Można powiedzieć: DataFrame to słownik Series o wspólnym indeksie.

Cecha	Series	DataFrame
Wymiary	1D (jedna kolumna)	2D (wiersze × kolumny)
Etykiety	indeks wierszy	indeks wierszy + nazwy kolumn
Analogia	jedna kolumna w arkuszu	cały arkusz kalkulacyjny
Powstaje gdy	df['kolumna']	df[['k1','k2']] lub pd.DataFrame(...)

Tworzenie Series i DataFrame; kolumna DataFrame jest typu Series.

```
import pandas as pd
s = pd.Series([5.0, 4.0, 4.5], index=['Logika', 'Etyka', 'Statystyka'])
print(s)          # po lewej indeks (etykiety), po prawej wartości
print(type(s))    # <class 'pandas.core.series.Series'>

df = pd.DataFrame({'ocena': [5.0, 4.0], 'przedmiot': ['Logika', 'Etyka']})
print(type(df['ocena'])) # <class '...Series'> -> kolumna ramki to Series
```

3.2. Różnica między loc a iloc

Oba służą do wybierania wierszy/kolumn, ale używają innych „adresów”:

- loc — indeksowanie po ETYKIETACH (nazwach). Zakresy są DOMKNIĘTE (koniec włącznie).
- iloc — indeksowanie po POZYCJACH liczbowych 0,1,2,... Zakresy jak w Pythonie: koniec WYŁĄCZNIE.

Porównanie loc (etykiety, koniec włącznie) i iloc (pozycje, koniec wyłączenie).

```
import pandas as pd
df = pd.DataFrame(
    {'imie': ['Adam', 'Beata', 'Cezary'], 'wiek': [20, 22, 21]},
    index=['a', 'b', 'c']) # własne etykiety wierszy: 'a','b','c'

# --- loc: po ETYKIETACH ---
df.loc['b']          # cały wiersz o etykiecie 'b' (Beata)
df.loc['a':'c', 'imie'] # od 'a' do 'c' WŁĄCZNIE, kolumna 'imie'
df.loc[df['wiek'] > 20, 'imie'] # loc przyjmuje też maskę logiczną (WHERE)

# --- iloc: po POZYCJACH (numerach) ---
df.iloc[0]          # pierwszy wiersz (pozycja 0) = Adam
df.iloc[0:2, 0]      # wiersze 0 i 1 (2 WYŁĄCZNIE), kolumna numer 0
df.iloc[-1]         # ostatni wiersz (Cezary)
```

Uwaga: Najczęstsza pomyłka: df.loc[0:2] przy domyślnym indeksie liczbowym zwraca wiersze 0,1,2 (3 sztuki), a df.iloc[0:2] zwraca tylko 0,1 (2 sztuki).

3.3. Podstawowe informacje o tabeli

Najważniejsze metody diagnostyczne ramki danych.


```
df.head(5)      # pierwsze 5 wierszy (podgląd danych)
df.tail(3)      # ostatnie 3 wiersze
df.shape        # krotka (liczba_wierszy, liczba_kolumn)
df.columns      # nazwy kolumn
df.dtypes       # typ danych każdej kolumny (int64, float64, object...)
df.info()       # podsumowanie: typy, liczba wartości niepustych, pamięć
df.describe()   # statystyki kolumn liczbowych: count, mean, std, min, kwartyle, max
df.isnull().sum() # liczba braków (NaN) w każdej kolumnie
```

Przykład z repozytorium (P2Lab06) — head i describe na wygenerowanych danych:

Generowanie danych i szybki przegląd (P2Lab06_NumPy_Pandas.ipynb).

```
import numpy as np, pandas as pd
np.random.seed(42)                # powtarzalność losowania
wiek = np.random.randint(18, 65, 100) # 100 liczb 18..64
dochód = np.random.normal(5000, 1500, 100) # rozkład normalny
df = pd.DataFrame({'wiek': wiek, 'dochód': dochód})
print(df.head())                  # podgląd pierwszych wierszy
print(df.describe())              # statystyki opisowe obu kolumn
```

4. NumPy: kształt obiektu ndarray i jego zmiana

NumPy to fundament obliczeń numerycznych w Pythonie. Jego główny typ to **ndarray** (N-wymiarowa tablica) — siatka elementów **tego samego typu**, ułożona w ciągłym bloku pamięci. Dzięki temu operacje na całych tablicach są bardzo szybkie. „Kształt” (shape) opisuje, ile elementów ma tablica w każdym wymiarze.

4.1. Jak sprawdzić kształt tablicy

Atrybuty opisujące tablicę: *shape*, *ndim*, *size*, *dtype*.

```
import numpy as np
x = np.arange(0, 12)          # wektor 0,1,2,...,11 (12 elementów)
x = x.reshape(3, 4)          # przekształć na 3 wiersze i 4 kolumny

x.shape    # (3, 4)  -> krotka: (liczba_wierszy, liczba_kolumn)
x.ndim     # 2       -> liczba wymiarów (osi)
x.size     # 12      -> łączna liczba elementów (3*4)
x.dtype    # dtype('int64') -> typ przechowywanych liczb
```

Uwaga: shape to zawsze KROTKA. Dla wektora (1D) ma jeden element, np. (12,) — przecinek odróżnia krotkę jednoelementową od zwykłego nawiasu.

4.2. Jak zmienić kształt tablicy

Służy do tego metoda `.reshape()`. Warunek: iloczyn nowych wymiarów musi równać się liczbie elementów (size). Przykład z repozytorium (P2Lab04_NumPy):

reshape, użycie -1, spłaszczanie i transpozycja.

```
x = np.arange(0, 12).reshape(3, 4)  # 12 liczb -> 3 x 4
x.reshape(2, 6)                     # te same 12 liczb -> 2 x 6

# -1 oznacza: 'policz ten wymiar sam'. Tu: 12/4 = 3 wiersze.
x.reshape(-1, 4)

x.ravel()    # spłaszcza do 1D (wektor 12-elementowy)
x.T          # transpozycja: zamienia wiersze z kolumnami (kształt 4 x 3)
```

Uwaga: reshape zwykle zwraca WIDOK (referencję) tych samych danych, a nie kopię — patrz sekcja 6.

5. NumPy: broadcasting, wektoryzacja, wydajność, axis

5.1. Broadcasting (rozgłaszanie)

Broadcasting to mechanizm pozwalający wykonywać operacje na tablicach o RÓŻNYCH kształtach bez ręcznego ich powielania. NumPy „rozciąga” mniejszą tablicę wzdłuż brakującego wymiaru, tak jakby ją skopiował — ale bez faktycznego zużywania pamięci. Reguła: wymiary porównuje się od prawej; pasują, gdy są równe albo jeden z nich wynosi 1.

Broadcasting wektora na macierz (przykład z P2Lab04).

```
import numpy as np
x = np.array([[2, 3],
              [4, 5]])      # kształt (2, 2)
y = np.array([1, 2])      # kształt (2,)

# y jest 'rozgłaszane' na każdy wiersz x: do każdego wiersza dodaje [1, 2]
y + x
# wynik:
# [[3, 5],
#  [5, 7]]
```

Skalar jako najprostszy przypadek broadcastingu.

```
# Klasyczny przykład: skalar rozgłasza się na całą tablicę
a = np.array([10, 20, 30])
a * 2          # [20, 40, 60] -> 2 'rozciąga się' na każdy element
a + 100        # [110, 120, 130]
```

5.2. Wektoryzacja

Wektoryzacja to wykonywanie operacji na **całej tablicy naraz**, jedną instrukcją, zamiast pisać pętlę po elementach. Kod jest krótszy i wielokrotnie szybszy, bo właściwa pętla wykonuje się w skompilowanym C wewnątrz NumPy, a nie w wolnym Pythonie.

Ta sama operacja: pętla kontra wersja zwektoryzowana.

```
import numpy as np
x = np.arange(1_000_000)

# Podejście NIEzwektoryzowane (wolne) – pętla w Pythonie:
# wynik = [v**2 for v in x]

# Podejście ZWEKTORYZOWANE (szybkie) – jedna operacja na całej tablicy:
wynik = x ** 2
```

Normalizacja min-max z repozytorium to także wektoryzacja — odejmowanie i dzielenie dzieją się na całym wektorze jednocześnie:

Wektorowa normalizacja min-max (P2Lab04_NumPy.ipynb).

```
v = np.array([10, 20, 30, 40, 50])
v_norm = (v - v.min()) / (v.max() - v.min()) # cała operacja na raz
# v_norm = [0., 0.25, 0.5, 0.75, 1.]
```

5.3. Dlaczego obliczenia w NumPy są wydajne

- Ciągła pamięć — elementy leżą obok siebie w jednym bloku, więc procesor czyta je bardzo szybko (lepsze użycie pamięci podręcznej).
- Jednolity typ (dtype) — brak narzutu Pythonowych obiektów na każdą liczbę; tablica miliona int64 to po prostu 8 MB liczb.
- Pętle w C — operacje wykonują wstępnie skompilowane funkcje w języku C/Fortran, bez interpretera Pythona.
- Wektoryzacja sprzętowa (SIMD) — procesor potrafi przetwarzać kilka liczb jedną instrukcją.
- Brak kopiowania przy broadcastingu — oszczędność pamięci i czasu.

5.4. np.vectorize(math.sin)(x) kontra np.sin(x)

To pytanie sprawdza, czy rozumiemy różnicę między **prawdziwą** wektoryzacją a jedynie wygodnym „opakowaniem” pętli:

- `np.sin(x)` — funkcja UNIWERSALNA (ufunc) napisana w C; działa na całej tablicy naprawdę równolegle, jest szybka.
- `np.vectorize(math.sin)(x)` — to tylko PĘTLA Pythona w przebraniu: dla każdego elementu wywołuje zwykłą `math.sin`. Daje ten sam WYNIK, ale jest znacznie wolniejsza (NumPy sam ostrzega, że to udogodnienie składniowe, a nie optymalizacja).

Prawdziwa ufunc vs. opakowana pętla.

```
import numpy as np, math
x = np.linspace(0, math.pi, 5)

np.sin(x)                # ufunc w C – szybkie, prawdziwa wektoryzacja
np.vectorize(math.sin)(x) # pętla po elementach – ten sam wynik, ale wolno
# Wniosek: WYNIK identyczny, WYDAJNOŚĆ dramatycznie różna na rzecz np.sin.
```

5.5. Czym są osie (axis) i jak zmienia się wymiar

Oś (axis) to numer wymiaru, WZDŁUŻ którego wykonujemy operację. W tablicy 2D: `axis=0` biegnie w dół (po wierszach → wynik na kolumnę), `axis=1` biegnie w bok (po kolumnach → wynik na wiersz). Po agregacji (`sum`, `mean`...) wskazana oś **znika** z kształtu.

Działanie axis na macierzy 5x5 (przykład z P2Lab04).

```
import numpy as np
x = np.random.randint(1, 100, 25).reshape(5, 5) # macierz 5 x 5

np.mean(x, axis=0) # średnia w każdej KOLUMNIE (oś 0 znika -> wynik (5,))
np.sum(x, axis=1)  # suma w każdym WIERSZU   (oś 1 znika -> wynik (5,))
np.sum(x)          # bez axis: jedna liczba – suma wszystkich elementów
```

Reguła ogólna: dla tablicy o kształcie (2, 3, 4) wywołanie `np.fun(x, axis=i)` USUWA i-ty wymiar:

Wywołanie	Co znika	Kształt wyniku
<code>np.fun(x, axis=0)</code>	wymiar 0 (rozmiar 2)	(3, 4)
<code>np.fun(x, axis=1)</code>	wymiar 1 (rozmiar 3)	(2, 4)
<code>np.fun(x, axis=2)</code>	wymiar 2 (rozmiar 4)	(2, 3)

Uwaga: Jeśli dodamy `keepdims=True`, oś nie znika, lecz kurczy się do rozmiaru 1 (np. (2,3,4) -> (1,3,4) dla `axis=0`) — przydatne, by wynik dało się rozgłosić z powrotem na oryginał.

6. NumPy: referencje (widoki) kontra kopie

Kluczowe dla unikania trudnych błędów: niektóre operacje zwracają **widok** (referencję do tych samych danych w pamięci), a inne **kopię** (nowy, niezależny blok pamięci). Zmiana widoku zmienia oryginał; zmiana kopii — nie.

6.1. Na czym polega różnica

- Widok (view) — nowa „etykieta” na te same dane. Modyfikacja przez widok zmienia tablicę źródłową. Oszczędza pamięć i czas.
- Kopia (copy) — całkowicie niezależny duplikat danych. Modyfikacja kopii NIE wpływa na oryginał.

Wycinek to widok; `.copy()` tworzy niezależny duplikat.

```
import numpy as np
x = np.arange(6)          # [0 1 2 3 4 5]

# WYCINEK (slicing) zwraca WIDOK:
w = x[1:4]                # widok na elementy [1 2 3]
w[0] = 999                # zmieniamy widok...
print(x)                  # [0 999 2 3 4 5] -> ORYGINAŁ też się zmienił!

# Jawna KOPIA jest niezależna:
k = x[1:4].copy()
k[0] = -1                 # zmiana kopii...
print(x)                  # [0 999 2 3 4 5] -> oryginał bez zmian
```

Uwaga: Możesz sprawdzić, czy obiekt jest widokiem: atrybut `w.base` wskazuje na tablicę źródłową (dla kopii `base` jest `None`).

6.2. Które operacje dają widok, a które kopię

Zwracają WIDOK (referencję)	Zwracają KOPIĘ
proste wycinki, np. <code>x[1:4]</code> , <code>x[:, 0]</code>	indeksowanie listą / tablicą indeksów (fancy indexing): <code>x[[0,2,4]]</code>
<code>reshape</code> (gdzie się da bez przenoszenia danych)	indeksowanie logiczne (maska): <code>x[x > 5]</code>
transpozycja <code>x.T</code> , <code>ravel()</code> (zwykle)	jawne <code>.copy()</code> , <code>np.array(x)</code> , <code>flatten()</code>
zmiana kształtu przez <code>x.shape = (...)</code>	operacje arytmetyczne: <code>x + 1</code> , <code>x * 2</code> (tworzą nową tablicę)

Uwaga: Dobra praktyka: jeśli zamierzasz modyfikować fragment, a nie chcesz ruszać oryginału — wykonaj jawnie `.copy()`. Operacje arytmetyczne i tak zwracają nową tablicę, więc `x = x + 1` jest bezpieczne, ale `x[mask] = 0` modyfikuje w miejscu.

7. Czyszczenie i standaryzacja danych

7.1. Co to jest czyszczenie danych

Czyszczenie danych (data cleaning) to przygotowanie surowych danych do analizy i modelowania: naprawienie lub usunięcie błędów, braków i niespójności. Zasada „garbage in, garbage out” mówi, że nawet najlepszy model nauczy się bzdur, jeśli nakarmimy go złymi danymi. W praktyce to często najbardziej czasochłonny etap pracy z danymi.

7.2. Typowe operacje czyszczenia danych

- Obsługa braków (NaN/NULL) — usunięcie wierszy/kolumn (`dropna`) albo wypełnienie wartością (`fillna`): średnią, medianą, najczęstszą wartością.
- Usuwanie duplikatów — powtórzone wiersze (`drop_duplicates`).
- Poprawa typów — np. tekst '12' zamieniany na liczbę, daty parsowane do typu `datetime`.
- Ujednolicanie kategorii — 'TAK', 'tak', 'Tak' sprowadzane do jednej formy.
- Obsługa wartości odstających (outliers) — wykrywanie i ewentualne przycinanie skrajnych wartości.
- Kodowanie zmiennych kategorycznych — np. one-hot encoding (tekst -> kolumny 0/1).
- Skalowanie/standaryzacja cech liczbowych (patrz niżej).

Typowe operacje czyszczące w pandas.

```
import pandas as pd, numpy as np

df.isnull().sum()          # ile braków w każdej kolumnie
df = df.drop_duplicates()  # usuń powtórzone wiersze
df = df.dropna(subset=['dochód']) # usuń wiersze bez dochodu

# Wypełnij braki MEDIANĄ kolumny (odporna na wartości odstające):
df['wiek'] = df['wiek'].fillna(df['wiek'].median())
```

W scikit-learn braki uzupełnia się estymatorem SimpleImputer, co pozwala wpiąć tę operację w spójny potok (pipeline):

Uzupełnianie braków estymatorem SimpleImputer (jak w P2Lab09_housing).

```
from sklearn.impute import SimpleImputer
# strategy='median' -> brakujące wartości zastępuje medianą każdej kolumny
imputer = SimpleImputer(strategy='median')
imputer.fit(dane_liczbowe) # policz mediany (uczenie na danych)
X = imputer.transform(dane_liczbowe) # zastąp braki wyliczonymi medianami
```

7.3. Standaryzacja danych — na czym polega i po co

Standaryzacja przekształca każdą cechę tak, aby miała **średnią 0 i odchylenie standardowe 1**. Dla wartości x odejmujemy średnią μ i dzielimy przez odchylenie σ :

$$z = (x - \mu) / \sigma$$

Po co to robimy? Bo wiele algorytmów porównuje cechy „po równo”. Jeśli jedna cecha to dochód (rzędu tysięcy), a druga to wiek (rzędu dziesiątek), to bez skalowania dochód zdominuje obliczenia odległości i gradientów. Standaryzacja wyrównuje wpływ cech i przyspiesza zbieżność (perceptron, Adaline, regresja logistyczna, sieci, K-Means — wszystkie tego wymagają).

Standaryzacja estymatorem StandardScaler (P2W10_Klasyfikacje_scikit_learn.ipynb).

```
import numpy as np
from sklearn.preprocessing import StandardScaler

data = np.array([[160], [170], [180], [190]]) # np. wzrost w cm

scaler = StandardScaler() # estymator standaryzacji
scaler.fit(data)          # ETAP UCZENIA: policz średnią i wariancję
print(scaler.mean_[0])    # wyestymowana średnia = 175.0
print(scaler.var_[0])     # wyestymowana wariancja

scaled = scaler.transform(data) # ETAP STOSOWANIA: przelicz na z = (x-mean)/std
print(scaled)                # wartości wokół zera, odchylenie 1
```

Uwaga: Najważniejsza zasada poprawności: scaler UCZYMY (fit) wyłącznie na zbiorze TRENINGOWYM, a potem TYLKO przekształcamy (transform) zbiór testowy tymi samymi parametrami. Inaczej dojdzie do „przecieku” informacji z testu do treningu. W repozytorium widać to wzorcowo: `sc.fit_transform(X_train)` oraz `sc.transform(X_test)`.

Poprawna kolejność: fit na treningu, transform na teście.

```
sc = StandardScaler()
X_train_std = sc.fit_transform(X_train) # ucź na treningu I przekształć trening
X_test_std  = sc.transform(X_test)     # test TYLKO przekształć (bez fit!)
```

8. Klasyfikacja i regresja w scikit-learn

scikit-learn (sklearn) to biblioteka uczenia maszynowego. Każdy model ma ten sam, prosty interfejs: tworzymy estymator, uczymy go metodą `.fit(X, y)`, a potem przewidujemy metodą `.predict(X)`. X to macierz cech (wiersze = przykłady, kolumny = cechy), y to wektor odpowiedzi.

8.1. Na czym polega problem klasyfikacji

Klasyfikacja to przewidywanie **etykiety/kategorii** (wartości dyskretnej): np. czy mail to spam (tak/nie), albo który z trzech gatunków irysa. Model uczy się granicy decyzyjnej rozdzielającej klasy na podstawie cech. Gdy klas są dwie — to klasyfikacja binarna, gdy więcej — wieloklasowa.

8.2. Metody klasyfikacji w sklearn

Metoda (klasa)	Krótki opis
Perceptron	Najprostszy klasyfikator liniowy; szuka prostej/hyperpłaszczyzny rozdzielającej klasy.
LogisticRegression	Regresja logistyczna — zwraca prawdopodobieństwo przynależności do klasy.
SGDClassifier	Klasyfikatory liniowe uczone spadkiem gradientu (z <code>loss='squared_error'</code> działa jak Adaline).
KNeighborsClassifier	k najbliższych sąsiadów — klasa wybierana głosami najbliższych przykładów.
SVC (SVM)	Maszyna wektorów nośnych — szuka marginesu maksymalnie oddzielającego klasy.
DecisionTreeClassifier	Drzewo decyzyjne — ciąg pytań tak/nie na cechach.
RandomForestClassifier	Las losowy — zespół wielu drzew, głosowanie większościowe.

Wzorcowy schemat klasyfikacji z repozytorium — perceptron na zbiorze irysów:

Pełny przepływ klasyfikacji (P2W10_Klasyfikacje_scikit_learn.ipynb).

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()          # wbudowany zbiór: 150 kwiatów, 4 cechy
X, y = iris.data, iris.target        # X = cechy, y = gatunek (0/1/2)

# Podział 70% trening / 30% test; stratify zachowuje proporcje klas
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

sc = StandardScaler()                # standaryzacja (perceptron jej wymaga)
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.transform(X_test)

ppn = Perceptron(eta0=0.1, random_state=42) # eta0 = szybkość uczenia
ppn.fit(X_train_std, y_train)          # UCZENIE modelu
y_pred = ppn.predict(X_test_std)       # PREDYKCJA na danych testowych
print('Dokładność:', accuracy_score(y_test, y_pred)) # ocena jakości
```

8.3. Na czym polega regresja i jak ją policzyć

Regresja to przewidywanie wartości **ciągłej** (liczby), np. ceny domu czy temperatury. Model szuka funkcji najlepiej dopasowanej do danych. Regresja liniowa zakłada zależność $y = w \cdot x + b$ i dobiera wagę w oraz wyraz wolny b tak, by zminimalizować błąd (sumę kwadratów odchyleń). Przykład z repozytorium:

Regresja liniowa: model odkrywa zależność $y = 2x + 1$.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4]]) # cecha (kolumna)
y = np.array([3, 5, 7, 9])         # prawdziwa zależność:  $y = 2 \cdot x + 1$ 

model = LinearRegression()
model.fit(X, y)                     # model sam estymuje  $w$  i  $b$ 

print('waga w:', model.coef_[0])    # ~2.0
print('wyraz wolny b:', model.intercept_) # ~1.0
print(model.predict([[10]]))        # przewidywanie dla  $x=10 \rightarrow \sim 21$ 
```

Dla zależności nieliniowych można rozszerzyć cechy o potęgi (regresja wielomianowa) — w repozytorium realizuje to PolynomialFeatures (plik P2Wyk11_PolyRegression_Wizualizacja.py).

9. Uczenie z nadzorem i bez nadzoru

To dwa podstawowe paradygmaty uczenia maszynowego, różniące się tym, czy dysponujemy „poprawnymi odpowiedziami” (etykietami y).

9.1. Uczenie z nadzorem (supervised)

Mamy dane wejściowe X **oraz** znane prawidłowe odpowiedzi y . Model uczy się odwzorowania $X \rightarrow y$, a potem przewiduje y dla nowych X . To jak nauka z nauczycielem, który sprawdza odpowiedzi. Dwa główne zadania:

- Klasyfikacja — odpowiedź to kategoria (gatunek irysa, spam/nie-spam). Przykłady: Perceptron, LogisticRegression, SVC, RandomForest.
- Regresja — odpowiedź to liczba (cena domu). Przykłady: LinearRegression, regresja wielomianowa.

9.2. Uczenie bez nadzoru (unsupervised)

Mamy tylko dane X , **bez** etykiet. Model sam szuka ukrytej struktury — grupuje podobne przykłady albo upraszcza dane. To jak odkrywanie wzorców bez podpowiedzi. Główne zadania:

- Klasteryzacja (grupowanie) — dzielenie danych na grupy podobnych obiektów. Przykłady z repo: K-Means, DBSCAN.
- Redukcja wymiarów — sprowadzanie wielu cech do kilku najważniejszych, np. PCA (do wizualizacji/kompresji).

Przykład klasteryzacji K-Means z repozytorium (uwaga: NIE używamy y do uczenia):

K-Means — uczenie bez nadzoru (P2W10).


```

from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

X = load_iris().data[:, [2, 3]]          # tylko cechy, etykiety IGNORUJEMY
X_std = StandardScaler().fit_transform(X) # klasteryzacja liczy odległości

# n_clusters=3: prosimy o 3 grupy (w praktyce dobiera się metodą łokcia)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
etykiety = kmeans.fit_predict(X_std)     # przypisanie każdego punktu do grupy

```

	Z nadzorem	Bez nadzoru
Dane	X + etykiety y	tylko X
Cel	przewidzieć y dla nowych X	znaleźć strukturę/grupy
Zadania	klasyfikacja, regresja	klasteryzacja, redukcja wymiarów
Przykłady	Perceptron, LinearRegression	K-Means, DBSCAN, PCA

10. Miary błędu: MSE, RMSE, MAE, accuracy, FP/FN

Aby ocenić model, mierzymy, jak bardzo jego przewidywania $\hat{y}$ odbiegają od prawdziwych wartości y. Inne miary stosujemy w regresji (błąd liczbowy), a inne w klasyfikacji (trafność etykiet).

10.1. Miary błędu w regresji

Oznaczmy: y_i — wartość prawdziwa, $\hat{y}_i$ — przewidywana, n — liczba przykładów.

MSE — błąd średniokwadratowy (Mean Squared Error)

$$MSE = (1/n) \cdot \sum (y_i - \hat{y}_i)^2$$

Średnia z KWADRATÓW błędów. Kwadrat sprawia, że duże pomyłki są karane wyjątkowo mocno (błąd 2x większy daje 4x większą karę). Wada: jednostka jest „kwadratowa” (np. zł²), więc trudna w interpretacji.

RMSE — pierwiastek z MSE (Root Mean Squared Error)

$$RMSE = \sqrt{MSE} = \sqrt{(1/n) \cdot \sum (y_i - \hat{y}_i)^2}$$

Pierwiastek przywraca pierwotną jednostkę (np. zł), więc wynik jest interpretowalny: „średnio mylimy się o tyle”. Wciąż mocno reaguje na duże błędy (jest na nie wrażliwy).

MAE — średni błąd bezwzględny (Mean Absolute Error)

$$MAE = (1/n) \cdot \sum |y_i - \hat{y}_i|$$

Średnia z WARTOŚCI BEZWZGLĘDNYCH błędów. Traktuje wszystkie błędy proporcjonalnie i jest odporna na wartości odstające (outliers nie są podnoszone do kwadratu).

Miara	Jednostka	Reakcja na duże błędy	Kiedy używać
MSE	kwadratowa	bardzo silna (kara ²)	optymalizacja, gdy duże błędy są groźne

Miara	Jednostka	Reakcja na duże błędy	Kiedy używać
RMSE	oryginalna	silna	raportowanie wyniku w naturalnej jednostce
MAE	oryginalna	umiarkowana, liniowa	gdy w danych są wartości odstające

Różnice w skrócie: $RMSE \geq MAE$ zawsze; im bardziej RMSE przewyższa MAE, tym więcej w danych dużych, pojedynczych błędów. MSE i RMSE „nie lubią” outlierów, MAE jest na nie odporna.

Liczenie miar błędu w sklearn.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

mse = mean_squared_error(y_test, y_pred)      # MSE
rmse = np.sqrt(mse)                          # RMSE = pierwiastek z MSE
mae = mean_absolute_error(y_test, y_pred)     # MAE
print(f'MSE={mse:.2f} RMSE={rmse:.2f} MAE={mae:.2f}')
```

10.2. Dokładność (accuracy) — miara dla klasyfikacji

Dokładność to odsetek poprawnie sklasyfikowanych przykładów:

$$accuracy = \text{liczba trafnych predykcji} / \text{liczba wszystkich predykcji}$$

Dokładność klasyfikatora.

```
from sklearn.metrics import accuracy_score
# Udział poprawnych odpowiedzi, np. 0.95 = 95% trafień
print(accuracy_score(y_test, y_pred))
```

Uwaga: Dokładność bywa myląca przy nie zrównoważonych klasach. Jeśli 99% maili to nie-spam, model mówiący zawsze „nie-spam” ma 99% accuracy, a jest bezużyteczny. Dlatego patrzymy też na FP/FN.

10.3. Błędy fałszywie pozytywny (FP) i fałszywie negatywny (FN)

W klasyfikacji binarnej (klasa „pozytywna” = to, co wykrywamy, np. „chory”, „spam”) możliwe są cztery wyniki, zestawione w tzw. macierzy pomyłek:

	Prawda: POZYTYW	Prawda: NEGATYW
Model mówi POZYTYW	TP — trafienie	FP — fałszywy alarm
Model mówi NEGATYW	FN — przeoczenie	TN — poprawne odrzucenie

- Fałszywie pozytywny (False Positive, FP) — model TWIERDZI „pozytyw”, a naprawdę jest negatyw. Fałszywy alarm (np. zdrowy uznany za chorego, zwykły mail wpada do spamu).
- Fałszywie negatywny (False Negative, FN) — model TWIERDZI „negatyw”, a naprawdę jest pozytywny. Przeoczenie (np. chory uznany za zdrowego, spam przepuszczony do skrzynki).

Który błąd groźniejszy zależy od zastosowania: w diagnostyce raka FN (przeoczenie choroby) jest dużo groźniejszy niż FP. Stąd dodatkowe miary: precyzja = $TP / (TP + FP)$ oraz czułość (recall) = $TP / (TP + FN)$.

11. Przeuczenie (overfitting) i jak mu zapobiegać

11.1. Co to jest przeuczenie

Przeuczenie (overfitting) to sytuacja, w której model „nauczył się na pamięć” danych treningowych — łącznie z ich szumem i przypadkowymi zależnościami — zamiast uchwycić ogólną regułę. Skutek: świetny wynik na treningu, ale słaby na nowych, niewidzianych danych. Model **nie generalizuje**.

Objaw rozpoznawczy: duża różnica między jakością na zbiorze treningowym (bardzo dobra) a testowym (wyraźnie gorsza). Przeciwnieństwo to **niedouczenie** (underfitting) — model zbyt prosty, słaby na obu zbiorach.

Uwaga: Analogia: student, który wykuł na pamięć odpowiedzi do konkretnych zadań z przykładowego testu, dostanie 100% na nim, ale obleje egzamin z nowymi zadaniami.

11.2. Jak przeciwdziałać przeuczeniu

- Więcej danych treningowych — trudniej „zapamiętać” duży, różnorodny zbiór.
- Prostszy model — mniej parametrów/nizszy stopień wielomianu ogranicza zdolność do dopasowania szumu.
- Regularyzacja — kara za zbyt duże wagi (L1/Lasso, L2/Ridge), zniechęca model do skrajnych dopasowań.
- Walidacja krzyżowa (cross-validation) — wielokrotny podział danych, by rzetelnie ocenić generalizację.
- Wczesne zatrzymanie (early stopping) — przerwanie uczenia, gdy błąd na zbiorze walidacyjnym zaczyna rosnąć.
- Dropout (w sieciach neuronowych) — losowe wyłączanie części neuronów podczas treningu.
- Augmentacja danych — sztuczne powiększanie zbioru (np. obroty, przesunięcia obrazów).

Dropout pojawia się w repozytorium (P2Wyk11_SieciNeuronowe) — losowo wyłącza np. 20% neuronów, co „rozprasza wiedzę” sieci na więcej neuronów i zapobiega przeuczeniu:

Dropout jako regularyzacja w sieci dla zbioru MNIST.

```
from tensorflow.keras import layers, models
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)), # spłaszcza obraz 28x28 do wektora 784
    layers.Dense(128, activation='relu'), # warstwa ukryta 128 neuronów
    layers.Dropout(0.2), # losowo wyłącza 20% neuronów -> anty-overfitting
    layers.Dense(10, activation='softmax') # 10 wyjść = 10 cyfr (prawdopodobieństwa)
])
```

12. Podział danych: zbiór treningowy i testowy

Dzielimy dane na **treningowe** (model się na nich uczy) i **testowe** (sprawdzamy na nich jakość). Cel: oszacować, jak model poradzi sobie z **nowymi, niewidzianymi** danymi. Gdybyśmy oceniali model na tych samych danych, na których się uczył, nie wykrylibyśmy przeuczenia — model mógłby je po prostu pamiętać.

Typowy podział to 70–80% trening i 20–30% test. Kluczowa zasada: zbiór testowy „odkładamy na bok” i NIE używamy go do żadnych decyzji o modelu aż do samego końca.

Podział danych funkcją `train_test_split` (wzorzec z repozytorium).

```
from sklearn.model_selection import train_test_split

# test_size=0.3 -> 30% danych trafia do testu, 70% do treningu
# random_state -> ustala losowość, by podział był POWTARZALNY
# stratify=y -> zachowuje proporcje klas w obu zbiorach (ważne w klasyfikacji)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

Uwaga: Czasem wydziela się jeszcze trzeci zbiór — WALIDACYJNY — do strojenia hiperparametrów. Wówczas: trening uczy wagi, walidacja wybiera ustawienia, test daje ostateczną, uczciwą ocenę.

13. Perceptron i funkcje aktywacji

13.1. Jak działa perceptron

Perceptron to najprostszy model „sztucznego neuronu” i podstawowy klasyfikator liniowy. Działa w trzech krokach:

- 1. Suma ważona — mnoży każde wejście x_i przez wagę w_i i sumuje, odejmując próg θ (bias): $z = w \cdot x - \theta$.
- 2. Funkcja aktywacji — przepuszcza z przez funkcję progową: wynik 1, jeśli $z \geq 0$, w przeciwnym razie 0.
- 3. Uczenie — porównuje wynik $\hat{y}$ z prawdą y i koryguje wagi proporcjonalnie do błędu.

Regułę korekty wag (z wykładu) zapisujemy tak — η to współczynnik szybkości uczenia, y wartość docelowa, $\hat{y}$ aktualny wynik perceptronu, x_i j -te wejście:

$$\Delta w_j = \eta \cdot (y - \hat{y}) \cdot x_j$$

Jeśli perceptron trafił ($y = \hat{y}$), poprawka jest zerowa. Jeśli się pomylił, wagi przesuwają się w stronę zmniejszającą błąd. Implementacja z repozytorium (P2Wyk11_SieciNeuronowe):

Rdzeń perceptronu: suma ważona + aktywacja progowa.

```
import numpy as np
class Perceptron(object):
    def __init__(self, input_size, lr=0.2, epochs=4):
        self.W = np.array([0.3, -0.2]) # początkowe wagi
        self.lr = lr # szybkość uczenia (eta)
        self.epochs = epochs # liczba przejść po danych
        # funkcja aktywacji: skok jednostkowy (0 lub 1)
        self.activation_function = lambda x: 0 if x < 0 else 1

    def predict(self, x, theta):
        z = self.W.T.dot(x) - theta # suma ważona minus próg
        a = self.activation_function(round(z, 2)) # przepuść przez aktywację
        return a # zwróć 0 lub 1
```

Uwaga: Perceptron z aktywacją zero-jedynkową może NIE zbiegać, jeśli dane nie są liniowo separowalne — wagi mogą się bez końca „przełączać”. Dane irysów (3 klasy) nie są w pełni liniowo separowalne, więc nie da się osiągnąć 100% trafności.

13.2. Co to jest funkcja aktywacji

Funkcja aktywacji decyduje, jak neuron przekształca swoją sumę ważoną z na sygnał wyjściowy. Jej najważniejsza rola: wprowadza **nieliniowość**. Bez niej cała sieć — choćby wielowarstwowa — byłaby równoważna jednemu przekształceniu liniowemu i nie nauczyłaby się złożonych zależności (np. funkcji XOR). Dodatkowo funkcja aktywacji musi być **różniczkowalna**, by zadziałała wsteczna propagacja błędów (sekcja 14).

13.3. Typowe funkcje aktywacji

Funkcja	Wzór	Charakterystyka
Skok jednostkowy	1 gdy $z \geq 0$, inaczej 0	klasyczny perceptron; nieróżniczkowalna
Liniowa (Adaline)	$\phi(z) = z$	wyjście wprost = suma ważona; różniczkowalna
Sigmoid	$1 / (1 + e^{-z})$	ściska do (0,1); daje prawdopodobieństwo (regresja log.)
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	ściska do (-1,1); wyśrodkowana w zerze
ReLU	$\max(0, z)$	szybka, popularna w sieciach głębokich
Softmax	$e^{z_i} / \sum e^{z_n}$	zamienia wektor na prawdopodobieństwa klas (wyjście)

Z repozytorium — sigmoid i jej pochodna (potrzebna do uczenia sieci XOR), oraz definicje ReLU i softmax z wykładu:

Sigmoid, jej pochodna i ReLU (P2Wyk11_SieciNeuronowe.ipynb).

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))          # ściska dowolne x do przedziału (0,1)

def sigmoid_derivative(y):
    # pochodna sigmoidu wyrażona przez jej WYJŚCIE y: s'(x) = s(x)*(1 - s(x))
    return y * (1 - y)

def relu(x):
    return np.maximum(0, x)              # zeruje wartości ujemne, dodatnie zostawia
```

Adaline (ADaptive LInear NEuron) to wariant perceptronu z LINIOWĄ aktywacją $\phi(z)=z$. Dzięki temu funkcja błędów jest różniczkowalna i wagi można uczyć spadkiem gradientu:

$$J(w) = \frac{1}{2} \cdot \sum_i (y^i - \phi(z^i))^2$$

14. Wsteczna propagacja błędów (backpropagation)

Pojedynczy neuron (perceptron) potrafi rozdzielać tylko dane liniowo separowalne — nie nauczy się np. funkcji XOR. Rozwiązaniem jest sieć **wielowarstwowa** (warstwa wejściowa, jedna lub więcej warstw ukrytych, warstwa wyjściowa). Problem: jak ustawić wagi w warstwach ukrytych, skoro nie znamy ich „prawidłowych” wartości? Odpowiedzią jest backpropagation.

14.1. Czemu służy propagacja błędów

Wsteczna propagacja błędów to algorytm **uczenia** sieci wielowarstwowej: efektywnie oblicza, jak każda waga w sieci wpływa na końcowy błąd, i wskazuje, w którą stronę ją poprawić, aby błąd zmalał. Jest to sposób na rozdzielenie „winy” za błąd pomiędzy wszystkie wagi, łącznie z tymi

głęboko w warstwach ukrytych.

14.2. Jak to działa (w skrócie)

- 1. Propagacja w przód (forward pass) — dane wejściowe przechodzą przez kolejne warstwy aż do wyjścia; sieć generuje predykcję $\hat{y}$.
- 2. Obliczenie błędu — porównujemy $\hat{y}$ z prawdziwą wartością y funkcją straty (np. MSE), otrzymując jedną liczbę — koszt.
- 3. Propagacja wstecz (backward pass) — błąd „cofa się” od wyjścia do wejścia. Korzystając z REGUŁY ŁAŃCUCHOWEJ liczenia pochodnych, wyznaczamy gradient (pochodną) kosztu względem KAŻDEJ wagi.
- 4. Aktualizacja wag — każdą wagę przesuwamy w kierunku PRZECIWNYM do gradientu (spadek gradientu): $w \leftarrow w - \eta \cdot \partial J / \partial w$.
- 5. Powtarzanie — kroki 1-4 powtarzamy przez wiele epok, aż błąd przestanie maleć.

Dlatego funkcje aktywacji muszą być różniczkowalne — reguła łańcuchowa wymaga pochodnych na każdym etapie. Aktualizacja wagi to: $\text{nowa_waga} = \text{stara_waga} - \eta \cdot \text{gradient}$.

$$w_j \leftarrow w_j - \eta \cdot \partial J / \partial w_j$$

Fragment z repozytorium — sieć dwuwarstwowa ucząca się XOR. Widać forward pass, liczenie błędu na wyjściu i cofanie go do warstwy ukrytej przez pochodne (sigmoid_derivative):

Wsteczna propagacja błędu w sieci uczącej się XOR (P2Wyk11_SieciNeuronowe.ipynb).

```
# Krok 1: FORWARD — sygnał płynie wejście -> warstwa ukryta -> wyjście
hidden = sigmoid(np.dot(X, self.W1) + self.b1)      # aktywacje warstwy ukrytej
output = sigmoid(np.dot(hidden, self.W2) + self.b2) # predykcja sieci

# Krok 2: BŁĄD na wyjściu (różnica prawda - predykcja)
error = y - output

# Krok 3: BACKWARD — delta = błąd * pochodna aktywacji (reguła łańcuchowa)
d_output = error * sigmoid_derivative(output)      # ile 'winy' na wyjściu
error_hidden = d_output.dot(self.W2.T)            # cofnij błąd do warstwy ukrytej
d_hidden = error_hidden * sigmoid_derivative(hidden) # delta warstwy ukrytej

# Krok 4: AKTUALIZACJA wag w stronę malejącego błędu (lr = szybkość uczenia)
self.W2 += hidden.T.dot(d_output) * self.lr
self.W1 += X.T.dot(d_hidden) * self.lr
```

Uwaga: Backpropagation to po prostu skuteczny sposób policzenia gradientu funkcji kosztu po wszystkich wagach. Samego „uczenia” (przesuwania wag) dokonuje spadek gradientu, a w nowoczesnych sieciach jego ulepszona wersja — optymalizator Adam (adaptacyjna szybkość uczenia dla każdej wagi).